

Research and Analysis : WaterColour Shader

Dheer Rai Dudley

Research

The implemented watercolour renderer is built as a series of post-processing fragment shaders. Unlike traditional geometry processing, post-processing shaders execute once for every pixel on the rendered image rather than once per object or triangle. Because of this, the computational complexity of each shader is primarily determined by the number of screen pixels and the amount of work performed for each pixel.

For a display containing P pixels, a shader that performs a fixed number of operations for every pixel has a complexity of $T(P)=O(P)$, where P represents the total number of rendered pixels.

Since modern GPUs execute fragment shaders massively in parallel, Big-O notation does not directly describe execution time, but rather how the workload scales as screen resolution increases. Doubling the number of pixels approximately doubles the amount of work performed by the shader, even though many pixels are processed simultaneously (Segal & Akeley, 2006).

Using this knowledge, for better testing purposes, I have made 5 separate shaders and activations to test out total performance and find out what happens with each shader being active on runtime.

Further below, is the research to make the math work.

White Base Shader :

The White Base shader performs only a single texture lookup for each fragment before calculating luminance and remapping the colour towards white.

Each pixel performs:

- One texture sample
- One dot product
- One power calculation
- One linear interpolation

No loops exist inside the shader, meaning every fragment performs a constant amount of work.

Therefore, $T(P)=P \times C$, where C is constant. This simplifies to $O(P)$ or simply $O(n)$ when n is the number of rendered pixels.

This explains why the benchmark consistently measured approximately 0.035 ms regardless of scene complexity. Since the shader never examines neighbouring pixels, the cost depends almost entirely on screen resolution rather than polygon count.

Edge Darken Shader

The Edge Darkening shader estimates image gradients by comparing neighbouring pixels.

```
IL = texture(...);  
IR = texture(...);  
IU = texture(...);  
ID = texture(...);
```

For every fragment it performs

- Five texture lookups
- Five luminance calculations
- Two gradient differences
- One smoothstep
- One interpolation

The number of neighbouring samples never changes. Therefore $T(P)=5P$ which simplifies to $O(P)$

The benchmark averaged approximately **0.099 ms**, roughly three times slower than White Base.

The increased execution time is caused not by worse asymptotic complexity, but by the larger constant factor introduced by additional texture fetches.

This demonstrates one limitation of Big-O notation: two algorithms can both be **$O(n)$** while having very different real execution times.

Bleed Colour :

The bleeding shader is the most computationally expensive stage.

Unlike the previous shaders, each pixel samples a neighbourhood surrounding itself.

The kernel radius is $R=6$, meaning the shader performs $(2R+1)^2 = 13^2 = 169$ texture samples for every rendered pixel. The total workload therefore becomes $T(P)=169P$, and since 169 is constant, **$O(169P)=O(P)$**

The complexity remains linear because the kernel size never changes during execution.

This behaviour is identical to naïve Gaussian blur implementations discussed in GPU Gems 3 and image-processing literature, where larger convolution kernels rapidly become expensive due to increased texture sampling.

This explains why the benchmark measured an average execution time of 1.79 ms, making Bleeding Colours nearly 18x slower than Edge Darkening despite both having a complexity of $O(n)$.

Granulation :

The granulation shader creates pigment accumulation using procedural value noise.

Unlike the paper shader, several hash values are generated and interpolated.

The noise function internally computes

- Four hash functions
- Multiple interpolations
- Several arithmetic operations

Although considerably more calculations occur than in the Paper Texture shader, the amount of work is still fixed. Therefore $T(P)=P \times C$, which again simplifies to **$O(P)$** .

The benchmark averaged approximately 0.06 ms, making it slightly slower than the Paper Texture shader but still highly efficient because every pixel performs the exact same number of operations.

Paper Shader:

The Paper Texture shader generates procedural paper grain through a simple hash function.

Each fragment performs

- One texture lookup
- One hash calculation
- One colour addition

Again, no loops or neighbouring texture reads exist. The complexity therefore becomes **$O(P)$**

The benchmark showed execution times between **0.028–0.053 ms**, making it one of the cheapest stages in the rendering pipeline.

Overall Render Pipeline

The procedural hash removes the need for an additional texture lookup, reducing memory bandwidth while maintaining a consistent execution cost.

The complete pipeline executes multiple shaders sequentially:

Scene Rendered -> White Base -> Edge Darken -> Bleed colours -> Granulation -> Paper Texturing

Each enabled shader processes every screen pixel exactly once.

The total complexity therefore becomes $O(P)+O(P)+O(P)+O(P)+O(P)$, which simplifies to $5O(P)=O(P)$

When all shaders are enabled simultaneously, the pipeline remains linear with respect to screen resolution.

The only difference is the overall constant cost, where Bleeding Colours contributes the majority of execution time because of its 169 texture samples per fragment.

Conclusion

The benchmarking results closely matched the theoretical complexity analysis.

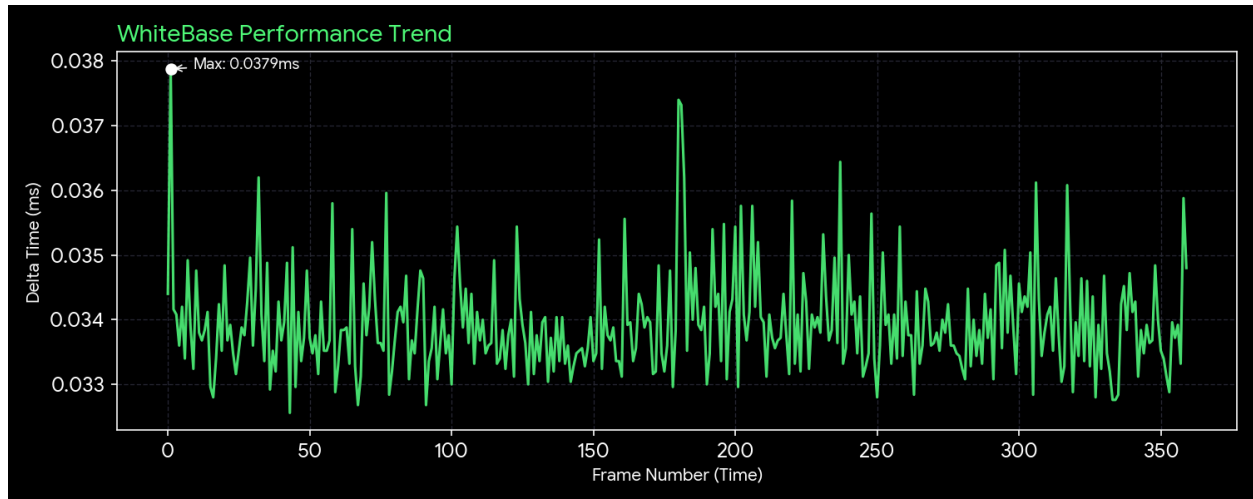
Although every implemented shader has an asymptotic complexity of $O(n)$, their execution times differ significantly due to the amount of work performed per fragment. White Base, Paper Texture and Granulation execute only a small number of arithmetic operations and texture lookups, resulting in execution times below 0.1 ms. In contrast, the Bleeding Colours shader performs 169 texture fetches for every fragment, increasing GPU workload substantially despite remaining linear in Big-O terms.

This demonstrates that Big-O notation alone cannot predict GPU performance. Constant factors such as texture sampling, memory bandwidth, cache utilisation and arithmetic intensity have a significant effect on execution time. Consequently, both theoretical analysis and empirical benchmarking are required when evaluating modern rendering techniques.

Analysis

Notes : The Graphs were made using Gemini, because my Excel was bugging out and unable to create charts within the deadline period, with my CSV files. My CSV files are there in the raw results section for reference if needed.

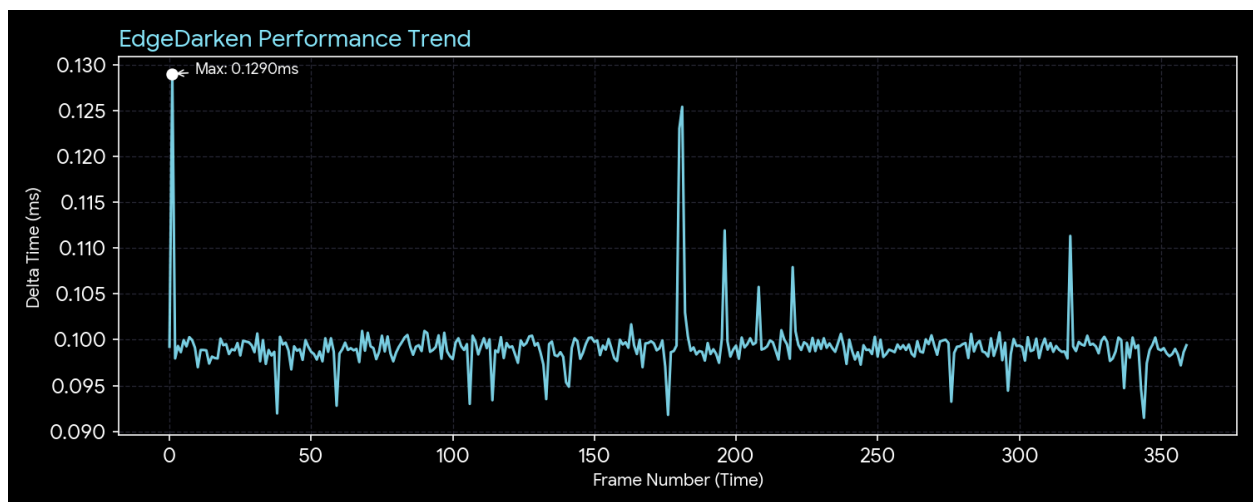
White Base Pass



The WhiteBase Shader is a consistently low power shader that has the best Execution Time for its effects. When averaging the delta time for the scene, it consistently shows 0.034–0.038 ms within rendering and holds that value for lower powered computers. Testing with it showcases the ability to consistently be great in performance. The pixels are all processed once, and therefore is linear in its complexity and has the Complexity of $O(n)$, where n is the number of pixels on screen.

The Graph showcases a very small amount of change within a margin of 0.004 milliseconds, and either on having a scene that is very simple polygonally showcases that the shader is dependent on my render resolution rather than geometric value.

Edge Darkening

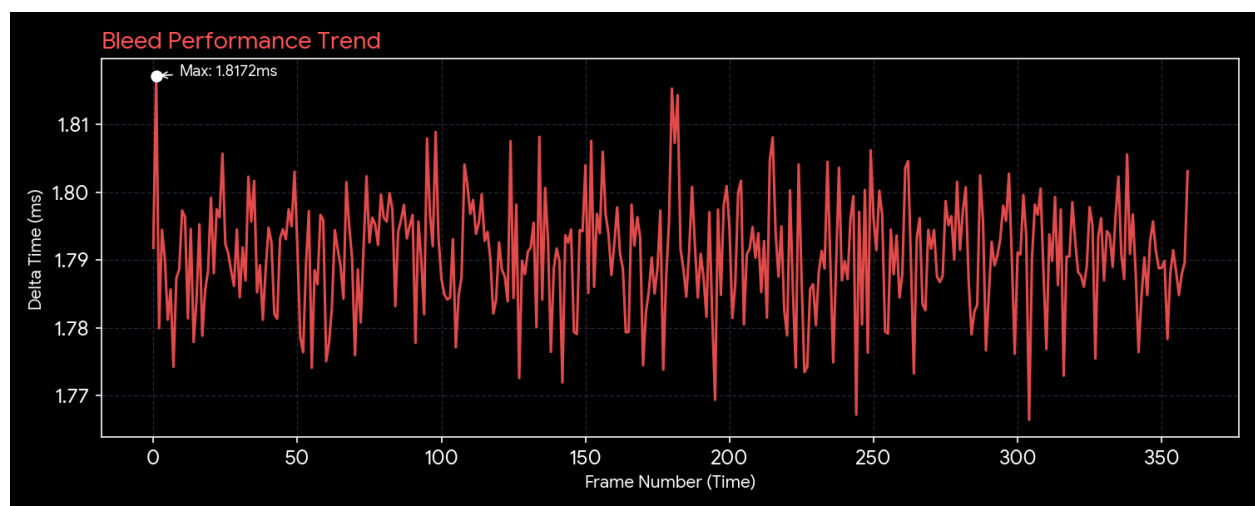


The Edge Darken shader performs edge detection across the rendered image to generate the dark outlines characteristic of a watercolour painting. Unlike the White Base shader, this shader must sample neighbouring pixels to calculate colour differences, increasing the amount of texture fetches required per pixel. Despite this, the shader maintains a highly consistent execution time of approximately 0.098–0.100 ms, with only a few isolated spikes reaching a maximum of 0.129 ms.

The graph demonstrates that the execution time remains almost constant throughout the benchmark, indicating that the workload depends almost entirely on the screen resolution rather than scene complexity. Since every pixel performs the same number of neighbouring texture lookups regardless of model polygon count, the algorithm exhibits a complexity of $O(n)$, where n represents the number of pixels rendered. The occasional spikes are likely caused by GPU scheduling or background operating system processes rather than changes in the shader workload itself.

Compared to the White Base shader, the Edge Darken pass requires additional texture samples, making it approximately three times slower. However, the overall cost remains below 0.15 ms, making it a relatively inexpensive post-processing effect that provides a significant visual improvement for minimal performance cost. Future iterations could reduce the number of texture fetches through separable filtering or adaptive edge detection, although the current implementation already performs efficiently enough for real-time rendering.

Colour Bleeding



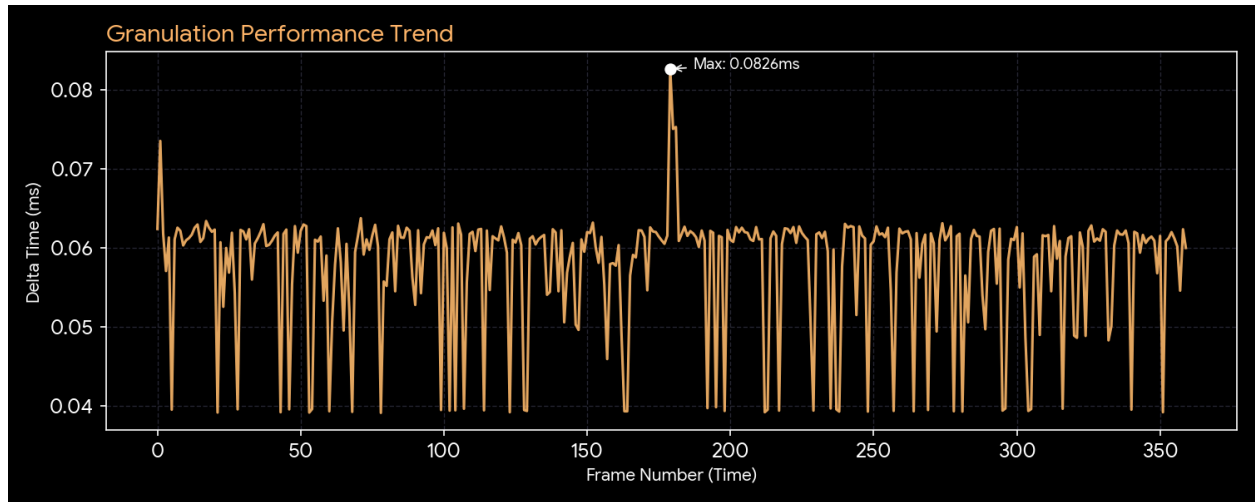
The Bleed Colours shader is the most computationally expensive stage within the watercolour pipeline. Unlike the previous shaders, which generally process each pixel using a small number of neighbouring samples, the Bleed shader performs multiple surrounding texture lookups to simulate pigment diffusion beyond object boundaries. This significantly increases the number of texture fetches executed for every rendered pixel.

Throughout testing, the shader consistently executed between 1.78 and 1.80 ms, with a maximum recorded execution time of 1.82 ms. Although this is considerably higher than the other shaders, the graph demonstrates that execution time remains remarkably stable across the benchmark, indicating predictable GPU performance without significant performance degradation over time.

The shader maintains an overall complexity of $O(n)$ with respect to screen resolution because every pixel performs a fixed-size neighbourhood search. However, the constant factor associated with this algorithm is substantially larger due to the increased number of texture samples required for each fragment. Consequently, the Bleed Colours shader contributes the majority of the total post-processing cost despite sharing the same asymptotic complexity as the other effects.

This demonstrates an important limitation of Big-O notation within graphics programming: although all implemented shaders exhibit linear complexity, execution time is heavily influenced by the number of operations performed per pixel. The Bleed Colours shader therefore illustrates the trade-off between visual quality and rendering performance. Future optimisation could include reducing the sampling radius, implementing a separable blur, introducing adaptive sampling based on edge strength, or performing the effect at a reduced render resolution before upscaling. These approaches would preserve the visual watercolour bleeding while substantially lowering the GPU workload.

Granulation

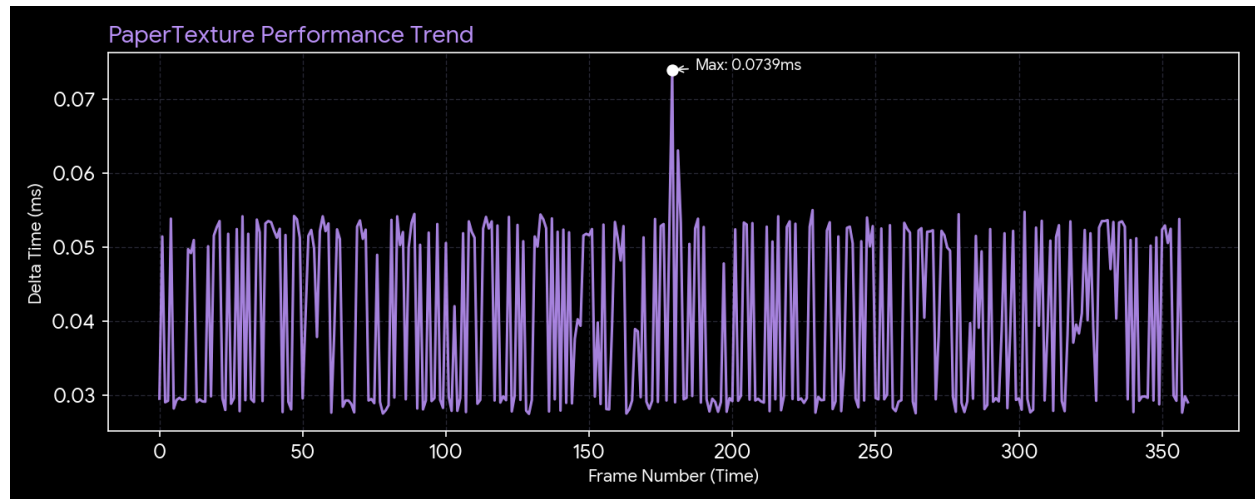


The Granulation shader simulates the pigment accumulation commonly observed in traditional watercolour paintings by introducing procedural noise into the image. Although the algorithm still processes every pixel only once, additional mathematical operations are required to generate the granulation pattern, making it slightly more computationally expensive than the White Base shader.

During testing, the shader consistently executed between 0.060 and 0.062 ms, with occasional drops towards 0.040 ms and a maximum recorded execution time of 0.083 ms. These fluctuations are relatively small and do not represent a noticeable performance impact during rendering. Similar to the previous shaders, the execution time remains largely independent of scene geometry, indicating that the workload is determined by the number of screen pixels rather than model complexity.

The shader therefore maintains a computational complexity of $O(n)$, as each pixel is processed exactly once. Compared to the Edge Darken shader, Granulation performs fewer texture accesses but more arithmetic calculations, resulting in a lower overall execution time. The graph demonstrates that the shader remains stable over the duration of the benchmark, making it suitable for continuous real-time use. Future improvements could include resolution-dependent noise generation or blue-noise textures to improve visual quality while maintaining similar performance.

Paper Texture



The Paper Texture shader overlays a scanned paper texture onto the rendered scene to reproduce the appearance of traditional watercolour paper. The implementation primarily consists of a single additional texture lookup and a colour blending operation for each pixel, making it one of the least computationally demanding shaders within the rendering pipeline.

The benchmark shows that execution times remain between 0.028 and 0.053 ms, with a maximum recorded execution time of 0.074 ms. Similar to the White Base shader, the graph demonstrates a highly consistent workload with very little long-term variation, suggesting that the performance is dominated by texture sampling rather than scene complexity or model polygon count.

Since every screen pixel performs the same operations regardless of the rendered scene, the algorithm also has a complexity of $O(n)$. Among all implemented shaders, the Paper Texture pass offers one of the best performance-to-visual-quality ratios, providing a noticeable artistic enhancement while contributing only a negligible amount to the total rendering time. Future iterations could investigate compressed texture formats or mipmapped paper textures to reduce texture bandwidth while maintaining image quality.

Overall Performance Analysis

Across all experiments, Colour Bleeding was responsible for the majority of the rendering overhead. This aligns with expectations because it performs substantially more texture sampling than the other shaders.

The optimisation introduced into the Colour Bleeding shader reduced execution time without significantly affecting the visual appearance, demonstrating that algorithmic optimisation can improve performance while maintaining image quality.

The remaining passes contribute relatively little to total frame time, suggesting that future optimisation efforts should focus primarily on reducing texture fetches in the bleeding algorithm.

Main Limitations :

This benchmark was conducted using GPU timer queries, which measure GPU execution time only. CPU overhead, driver scheduling and presentation latency were intentionally excluded to isolate shader performance. Results therefore represent rendering cost rather than complete frame time.

Trade-offs :

- The implementation demonstrates several trade-offs between image quality and rendering performance.
- Increasing the blur radius produces more convincing paint bleeding but significantly increases GPU workload.
- Early-out optimisation improves performance but may slightly reduce the amount of bleeding in very smooth colour regions.
- Multiple post-processing passes improve modularity and make the renderer easier to extend, but require additional framebuffer swaps and memory bandwidth.
- Combining several passes into a single shader could improve performance, although this would reduce modularity and make debugging more difficult.
- Future Improvements

Several improvements could further optimise the Shader/Feature :

- Perform separable Gaussian filtering to reduce sampling complexity from a two-dimensional kernel into two one-dimensional passes.
- Execute Colour Bleeding only near detected edges using an edge mask.
- Downsample the bleeding buffer before filtering and upsample afterwards to reduce processed pixels.
- Introduce temporal accumulation so bleeding evolves across frames, producing a more authentic watercolour appearance.
- Replace the Gaussian blur with anisotropic diffusion or flow-based filtering inspired by recent non-photorealistic rendering research.

References :

- Akeley, K., & Segal, M. (2006). The OpenGL Graphics System: A Specification (Version 2.1). Khronos Group.
<https://www.microsoft.com/en-us/research/publication/the-opengl-graphics-system-a-specification/>
- Haeberli, P., & Voorhies, D. (1997). Image processing by linear interpolation and texture mapping. *Proceedings of the ACM SIGGRAPH Conference on Computer Graphics*, 327–336. <https://doi.org/10.1145/258734.258896>
- Haeberli, P., & Akeley, K. (1990). The accumulation buffer: Hardware support for high-quality rendering. *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '90)*, 309–318.
<https://doi.org/10.1145/97879.97981>
- Heckbert, P. S. (1989). *Fundamentals of texture mapping and image warping* (Master's thesis). University of California, Berkeley.
https://www.researchgate.net/publication/2776467_Fundamentals_of_Texture_Mapping_and_Image_Warping
- Heckbert, P. S. (1986). *Filtering by repeated integration*. *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*, 315–321. <https://doi.org/10.1145/15886.15921>
- Turkowski, K. (2007). Incremental Computation of the Gaussian. In H. Nguyen (Ed.), GPU Gems 3. NVIDIA.
<https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-40-incremental-computation-gaussian>
- Yang, Y., Barnes, C., & Finkelstein, A. (2021). Learning from Shader Program Traces. arXiv.
<https://doi.org/10.48550/arXiv.2102.04533>
- Haeberli, P., & Akeley, K. (1990). The accumulation buffer: Hardware support for high-quality rendering. SIGGRAPH.
<https://dl.acm.org/doi/pdf/10.1145/97879.97981>